

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUA – PEC 1

Presentación

Esta primera PEC sirve como introducción a las arquitecturas de software para el desarrollo de aplicaciones distribuidas donde se trabajan los cimientos, técnicas y habilidades aplicables en la definición moderna de arquitecturas del software y se profundiza en la selección del estilo arquitectónico más adecuado según la tipología de la aplicación a desarrollar.

Para elaborar la actividad se formulan cuatro ejercicios: el primer ejercicio evaluará alguno de los aspectos fundamentales en la definición de una arquitectura, mientras que el segundo y el tercer ejercicio habrá que decidir y razonar el estilo arquitectónico (o mezcla de estilos) más adecuado en función de los requisitos planteados en un caso de estudio. Finalmente, el último ejercicio evaluará algunos conceptos básicos de las arquitecturas distribuidas.

La PEC 1 alcanza los contenidos del libro *Fundamentals of Software Architecture* (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PEC 1 se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

Los objetivos de la PEC 1 son:

- Explicar los fundamentos de la arquitectura de software y las características de una arquitectura.
- Exponer argumentos sobre los puntos fuertes y los puntos débiles de los diferentes estilos arquitectónicos y cómo estos influyen en la decisión del estilo o estilos arquitectónicos más adecuados para una determinada aplicación.

- Situar los nuevos paradigmas emergentes de la ingeniería del software dentro del marco de las arquitecturas del software.

Descripción de la PEC a realizar

Ejercicio 1 (2 puntos)

Realizad un estudio comparativo de los estilos arquitectónicos *Event Driven* y *Orchestration-Driven Service-Oriented* en función de las características arquitectónicas de cada uno de ellos. Usad la tabla a continuación.

Por último, proponed un ejemplo de aplicación que encaje en cada uno de los dos estilos arquitectónicos.

	<i>Event Driven</i>	<i>Orchestration-Driven Service-Oriented</i>
<i>Partitioning type</i>		
<i>Number of quanta</i>		
<i>Deployability</i>		
<i>Elasticity</i>		
<i>Evolutionary</i>		
<i>Fault tolerance</i>		
<i>Modularity</i>		
<i>Overall cost</i>		
<i>Performance</i>		
<i>Reliability</i>		
<i>Scalability</i>		
<i>Simplicity</i>		
<i>Testability</i>		

Solución

	<i>Event Driven</i>	<i>Orchestration-Driven Service-Oriented</i>
<i>Partitioning type</i>	Separación puramente tecnológica.	Separación puramente tecnológica.
<i>Number of quanta</i>	Dependiendo de la implementación pueden variar de uno a diversos.	Uno.
<i>Deployability</i>	Habilidad de desplegar versiones frecuentemente.	Poco soporte para ser desplegado fácilmente.

<i>Elasticity</i>	No es una de sus mejores características.	Buena elasticidad a pesar de la dificultad intrínseca de replicar la sesión en distintas instancias del servidor de aplicaciones.
<i>Evolutionary</i>	Excelente para añadir nuevas funcionalidades, ya sea a procesadores de eventos existentes o bien creando de nuevos.	Muy mala debido al acoplamiento con el punto central de orquestación.
<i>Fault tolerance</i>	Tolerancia a fallos máxima que se consigue intermediando <i>event processors</i> altamente desacoplados y asíncronos que proporcionan consistencia y procesamiento eventual de los <i>event workflows</i> .	Buena tolerancia a fallos puesto que se puede configurar el orquestador para limitar el acceso a componentes conflictivos.
<i>Modularity</i>	Buen modularidad.	Buena, se puede añadir funcionalidad nueva mediante nuevos componentes y conectándolos al orquestador.
<i>Overall cost</i>	Coste muy elevado.	Coste muy elevado al necesitar el orquestador y seguir las reglas de despliegue de componentes que marca.
<i>Performance</i>	Rendimiento máximo conseguido con comunicaciones asíncronas combinadas con procesamiento paralelo.	Rendimiento muy bajo por el hecho de necesitar siempre pasar por el orquestador para cualquier llamada.
<i>Reliability</i>	Algo menos <i>reliable</i> .	Fiabilidad menor debido a la cantidad de componentes que deben ser ejecutados para una acción sobre el sistema,
<i>Scalability</i>	Escalabilidad elevada mediante técnicas como los balanceos programáticos de los procesadores de eventos	Escalabilidad elevada incluyendo nuevos servidores de aplicación que pueden soportar más carga, a pesar del peaje de pagar el coste de la sincronización entre ellos y la replicación de sesión.
<i>Simplicity</i>	Muy compleja. La complejidad es uno de los <i>drawbacks</i> principales.	Muy compleja por la razón de que intervienen demasiados componentes de infraestructura no funcional.
<i>Testability</i>	Relativamente poco debido a la naturaleza no determinista y dinámica de los flujos de eventos	Las pruebas se realizan con dificultad en este complejo estilo.

Un ejemplo de aplicación que encaje en un estilo arquitectónico *Event Driven* podría ser un juego en que los personajes reaccionen a diferentes eventos.

Aunque este estilo es llamado por su valor histórico como precursor de los microservicios, un ejemplo de aplicación que encaje en un estilo arquitectónico *Orchestration-Driven Service-Oriented* podría ser una aplicación de gestión empresarial de un negocio consolidado con procesos maduros y bien definidos, con pocas perspectivas de crecimiento o variación de sus funcionalidades. Se pueden añadir más servidores de aplicaciones si la demanda crece y escalar la base de datos sólo verticalmente.

Ejercicio 2 (3 puntos)

Durante la época de pandemia muchos negocios se vieron obligados a reinventarse para no tener que desaparecer. Uno de ellos ha sido el mundo de la restauración. Un negocio familiar de restauración que cuenta con 3 pizzerías en la periferia de la ciudad de Madrid ha decidido empezar a realizar reparto a domicilio.

Para hacerlo, aparte de la atención telefónica, han decidido crear una aplicación web muy sencilla que tenga la carta de productos disponibles y permite a los clientes hacer el pedido. Para implementar la aplicación se ha contratado una empresa pequeña que cuenta con un equipo de desarrollo propio con 1 experto en UI y 2 desarrolladores PHP (uno de ellos con conocimientos de bases de datos relacionales). El miembro más experimentado del equipo de desarrollo hará también de arquitecto.

El despliegue de toda la aplicación se hará en un servidor Apache con un módulo PHP y los datos se persistirán en una base de datos relacional MySQL. Tanto el servidor PHP como la base de datos están instalados en una infraestructura "on-premise".

Por último, hay que tener en cuenta que la situación económica del grupo de restauración es bastante crítica, por eso nos interesa mucho más tener algo funcionando rápidamente y con un coste bajo, que pensar en posibles problemas de rendimiento, mantenibilidad o modularidad. Tampoco nos preocupa la escalabilidad de la aplicación (con 3 pizzerías no parece que vayamos a tener problemas es ese aspecto).

¿Qué estilo arquitectónico de los que habéis estudiado en el Capítulo II: *Architecture Styles* del libro *Fundamentals of Software Architecture* creéis que encaja más con la naturaleza del proyecto?

Razonad la respuesta comparando los requerimientos y restricciones del proyecto a desarrollar con los pros y contras del estilo arquitectónico elegido.

Solución

El estilo arquitectónico que encaja más para desarrollar el proyecto sería un estilo arquitectónico por capas (***Layered Architecture Style***). Algunos de los motivos su los siguientes:

- Según los requerimientos se trata de hacer una aplicación con acceso web muy sencilla, que hay que hacer con una tecnología conocida, con un coste bajo y disponible rápidamente. El estilo arquitectónico en capas está principalmente enfocado a resolver este tipo de aplicaciones: *"This style of architecture is the de facto standard for most applications, primarily because of its simplicity, familiarity, and low cost"*
- Teniendo en cuenta la composición del equipo de desarrollo, sin expertos en el dominio, la separación por capas sería una separación "tecnológica" y esta separación es precisamente la que tenemos en un estilo arquitectónico en capas: *"The layered architecture is a technically partitioned architecture (as opposed to a domain-partitioned architecture)"*
- La separación en capas encaja perfectamente con una topología basada en una interfaz web, un servidor de aplicaciones para la capa de negocio y un servidor de base de datos separado. Se trata de una topología típica en este estilo arquitectónico.
- El hecho que el despliegue sea *"on-premise"* también encaja en un estilo arquitectónico en capas, es el modelo típico por estas aplicaciones.

Algunos de los puntos débiles de estilo arquitectónico en capas son el modularidad, la escalabilidad, el rendimiento y la mantenibilidad, pero los requisitos nos explicitan que estas características son poco importantes en este estadio del proyecto.

Ved el Capítulo 10. *Layered Architecture Style* del libro *Fundamentals of Software Architecture* para una explicación más extensa de la arquitectura propuesta y como encaja en los requisitos del ejercicio.

Ejercicio 3 (3 puntos)

El grupo inversor que puso en marcha la tienda de material fotográfico ("**Photo&Film4You**") que os adjuntamos con este enunciado, logró poner en marcha la aplicación en un tiempo récord y esto les ha permitido superar con éxito las expectativas de negocio. No sólo esto, sino que la iniciativa ha tenido muy buena acogida; sus usuarios ya se cuentan por decenas de miles y cuentan con un catálogo de más de 1500 referencias. El éxito ha sido tal que, incluso, se plantean dar el salto al mercado americano y asiático.

Pero no todo es tan bonito, debido a este rápido crecimiento ya están empezando a encontrarse algunos problemas y nuevos retos con la aplicación actual de alquiler de equipamientos. Los enumeramos brevemente:

- Hay equipos en los que el número de usuarios que desea realizar la reserva es muy elevado, en este escenario la aplicación ya va muy lenta y los servidores "on-premise" van muy saturados. La elevada concurrencia comienza a ser un grave problema para la plataforma.
- La aplicación tiene periodos de uso muy intensivo en los que la infraestructura actual no es suficiente pero no nos podemos permitir dimensionar la infraestructura para esos periodos puntuales de alta demanda ya que estaríamos malgastando muchos recursos el resto de tiempo.
- Cada vez parece más claro que los requisitos tecnológicos no son homogéneos para toda la aplicación.

- Nos hemos encontrado con errores de regresión que nos han dejado la plataforma fuera de servicio durante horas, con la consiguiente pérdida de dinero que esto implica. Esto es debido a que no tenemos una batería de tests automáticos que nos asegure que las releases funcionarán correctamente y no romperán nada.
- Cada vez existen procesos más complejos y especializados que requieren un dominio más específico tanto a nivel funcional como a nivel técnico. Ya no es posible que un equipo de desarrollo pequeño domine todos estos conceptos. Existen procesos complejos que se resuelven mejor utilizando técnicas de programación funcional con lenguajes de programación funcionales y fuentes de datos NoSQL mientras que los procesos de facturación requieren ser implementados con tecnologías más tradicionales.
- Para ser competitivos necesitamos desarrollar nuevas funcionalidades y ponerlas en producción muy rápidamente, ¡si no lo hacemos nosotros lo hace la competencia! Idealmente, una funcionalidad debería promocionar a producción de forma segura sin intervención manual, y este ciclo podría repetirse varias veces al día. Esto último implica que los despliegues de las nuevas funcionalidades deben ser independientes y seguir su propio ciclo de vida.

¿Qué estilo arquitectónico de los que habéis estudiado al Capítulo II: *Architecture Styles* del libro *Fundamentals of Software Architecture* creéis que encaja con los nuevos requerimientos del proyecto?

Razonad la respuesta comparando los nuevos requerimientos y restricciones del proyecto a desarrollar con los pros y contras del estilo arquitectónico que elijáis.

Solución

El estilo arquitectónico hacia el que tiene que evolucionar el proyecto es un estilo arquitectónico basado en microservicios (**Microservices Architecture**). Algunos de los motivos son los siguientes:

- Una arquitectura basada en microservicios es altamente escalable y, desplegada en una plataforma adecuada al Cloud, permitirá solventar los problemas de rendimiento y de picos de uso con un coste razonable usando técnicas de autoescalado.
- Una arquitectura basada en microservicios permite utilizar la mejor tecnología para cada servicio y que estos colaboren entre ellos.
- Los microservicios pueden tener dominios y funcionalidades altamente especializadas, pero tienen que ser como una caja negra para el resto del sistema. Esto soluciona el problema de especialización mencionado, pasando de una separación puramente "tecnológica" de las capas a una separación por "conceptos de dominio" propia de las arquitecturas basadas en microservicios.
- Un microservicio evoluciona y se puede desplegar de forma independiente del resto de componentes del ecosistema. Esto permite desarrollar nuevas funcionalidades y ponerlas en producción muy rápidamente
- Una de las bases de una arquitectura basada en microservicios es la "testeabilidad", donde cada microservicio tiene su batería de *test* automáticos que asegurarán que no se introduzcan errores de regresión en los despliegues.

Algunos de los puntos débiles de estilo arquitectónico basado en microservicios son la complejidad, el rendimiento por la naturaleza distribuida y el coste en el desarrollo. También hay que mencionar que el uso eficiente de microservicios requiere una cultura DevOps madura y una metodología de desarrollo ágil.

Usad el Capítulo 17: *Microservices Architecture* del libro *Fundamentals of Software Architecture* para una explicación más extensa y detallada de la arquitectura propuesta y su encaje con las restricciones y los nuevos requerimientos del proyecto.

Ejercicio 4 (2 puntos)

Responded brevemente a las siguientes preguntas sobre los conceptos vistos en el libro "*Fundamentals of software architecture*".

4.1 (0.5 puntos) Elegid el top 3 de las falacias y consideraciones de la computación distribuida que consideréis que pueden tener más impacto en las aplicaciones. Argumentad porqué las habéis elegido y poned un ejemplo concreto para cada una de ellas. Los ejemplos elegidos no pueden aparecer en el libro "*Fundamentals of Software Architecture*".

Solución

Las falacias de la computación distribuida fueron propuestas por L. Peter Deutsch y otros colegas de Sun Microsystems en 1994. Describen algunas presunciones equivocadas sobre la computación distribuida que pueden complicar sobremanera el sistema si no se tienen en cuenta. A modo de solución de ejemplo, se escogen tres al azar y se justifican:

1. "La red es fiable".

A pesar de los avances en la tecnología, que se han producido en los últimos años, la red sigue siendo un factor poco fiable. Podemos tener dos sistemas perfectamente maduros y probados que, al añadir la red para comunicarse entre ellos, ésta introduce latencias y caídas que hacen que la globalidad del sistema baje en fiabilidad.

2. "El ancho de banda es infinito"

Como en todo sistema físico, y la red no está exenta, los recursos son finitos. Al diseñar sistemas distribuidos, no sólo deben dimensionarse los servidores participantes, sino también la red de comunicaciones entre ellos. Por ejemplo, gestionar un pico de consumo de Black Friday sólo puede realizarse si se contempla el ancho de banda necesario para que los consumidores puedan llegar a la tienda web.

3. "La topología de la red nunca cambia"

De la misma forma que cambian los sistemas y las aplicaciones con nuevas funcionalidades, también cambian las redes y no siempre lo hacen teniendo en cuenta las aplicaciones que se ejecutan sobre

ella. Por ejemplo, la introducción de elementos de seguridad en las redes puede incidir gravemente en la latencia entre los sistemas, factor que puede ser fatal en alguno de ellos o sus componentes.

Véase el “*Capítulo 9: Foundations*” del libro “*Fundamentals of Software Architecture*”.

4.2 (0.5 puntos) Cuál de estas tareas asociadas son responsabilidades de un arquitecto y qué son responsabilidad del equipo de desarrollo:

- Realizar un diagrama de estados por los que pasa un pedido.
- Crear un prototipo con la interfaz de usuario para verificar la experiencia de los usuarios con la aplicación antes de desarrollarla
- Definir como se gestiona a nivel de infraestructura el incremento de uso durante las campañas de marketing.
- Automatización de las pruebas
- Definir la mejor tecnología para implementar los procesos

Solución

- Realizar un diagrama de estados por los que pasa un pedido: **Equipo de desarrollo.**
- Crear un prototipo con la interfaz de usuario para verificar la experiencia de los usuarios con la aplicación antes de desarrollarla: **Equipo de desarrollo.**
- Definir como se gestiona a nivel de infraestructura el incremento de uso durante las campañas de marketing: **Arquitecto.**
- Automatización de las pruebas: **Equipo de desarrollo.**
- Definir la mejor tecnología para implementar los procesos: **Arquitecto.**

Ved el *Capítulo 2. “Architectural Thinking - Architecture Versus Design”* del libro “*Fundamentals of Software Architecture*”.

4.3 (0.5 puntos) Poned un ejemplo concreto para una aplicación que siga un estilo arquitectónico basado en microservicios donde se vea que no es posible maximizar TODAS las características deseables para esta arquitectura.

Solución

No es posible maximizar TODAS las características deseables para una arquitectura puesto que muchas veces estas características entran en contradicción y se tiene que llegar a un compromiso en el grado de compleción de cada una de ellas.

Hay muchos ejemplos, todos ellos aplicables a los distintos estilos arquitectónicos que habéis estudiado. Un ejemplo muy típico, aplicable a cualquier estilo, es la seguridad y el rendimiento, contra más seguridad exigimos menos rendimiento tendrá el sistema.

Un ejemplo también típico y muy aplicable para arquitecturas basadas en microservicios es la simplicidad i el coste: Una arquitectura basada en microservicios será inherentemente compleja i cuanto más aumente esa complejidad mayo coste tendremos (en todos los sentidos).

Ved el Capítulo 4. *Architecture Characteristics Defined* del libro *Fundamentals of Software Architecture*.

4.4 (0.5 puntos) Elegid dos expectativas que creáis que están asociadas al rol de arquitecto que se hayan debatido en el debate de la actividad y poned un ejemplo de tarea concreta por cada una de ellas. Los ejemplos elegidos no pueden aparecer en el libro *Fundamentals of Software Architecture*.

Solución

- **Analizar continuamente la arquitectura:** *Evaluar las alternativas de despliegue al Cloud para ver si hay que hacer una migración en este sentido.*
- **Tener cierto dominio del negocio:** *En el negocio bancario, conocer qué es una hipoteca inversa.*

Ved el Capítulo 1: *Introduction* del libro *Fundamentals of Software Architecture*.

Recursos

Recursos Básicos

- Libro "Fundamentals of Software Architecture" (Richards & Ford): Chapter 1. Introduction.
- Libro "Fundamentals of Software Architecture" (Richards & Ford): I Foundations (Chapters 2, 4, 5 y 8)
- Libro "Fundamentals of Software Architecture" (Richards & Ford): II Architecture Styles (Chapters 9 - 14, 16 - 18)

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que enviar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PEC1.pdf*.

Este documento se enviará al espacio de “Entrega de la PEC 1” del aula antes de las **23:59 horas del día 02 de abril de 2024**. No se aceptarán entregas fuera de plazo.